

RGAT com mais épocas de treino: revisão da hipótese de sub-treino

1 Motivação

`estudo/scibench_ablation.tex` concluiu, a partir de quatro comparações controladas (2 tamanhos de modelo $\times$ 2 benchmarks, todas com 3 épocas de treino), que o RGAT nunca supera sua contraparte T5 pura do mesmo tamanho. Antes de aceitar essa conclusão como definitiva, testamos a hipótese mais óbvia de confusão: será que 3 épocas simplesmente não são treino suficiente para uma arquitetura que adiciona camadas de atenção sobre grafo inicializadas do zero (o RGAT não tem pré-treino próprio, ao contrário do backbone T5)? Se o RGAT precisa de mais gradiente para as camadas novas convergirem, o resultado anterior mediria sub-treino, não uma limitação real da arquitetura.

Para testar isso, re-treinamos as duas configurações com RGAT no ScienceBenchmark por **10 épocas** (mesmos dados, mesmos hiperparâmetros de *learning rate*/otimizador/*batch* efetivo das rodadas de 3 épocas – único fator alterado é `num_train_epochs`):

- **t5-small + RGAT:** `graphix-scibench-t5small-rgat-10ep`
- **t5-base + RGAT completo** (+oncomx, *gradient checkpointing*): `graphix-scibench-finetune-gc-10ep`

2 Resultado principal

Table 1: Comparação 3 épocas vs. 10 épocas (melhor checkpoint por `eval_loss`), dev set do ScienceBenchmark.

Configuração	<code>eval_loss</code>	<code>exact match</code>	<code>exec</code>	tempo de treino
t5-small + RGAT, 3 épocas	1,1517	1,04%	0,68%	3h46min
t5-small + RGAT, 10 épocas	0,9243	5,19%	4,10%	6h59min
t5-base + RGAT completo, 3 épocas	1,6765	0,0%	0,34%	9h42min
t5-base + RGAT completo, 10 épocas	1,4567	0,0%	2,73%	16h13min

O achado é inequívoco: mais épocas melhoram substancialmente o RGAT em ambos os tamanhos de modelo. A Figura 1 mostra que a melhora não satura nas 3 épocas originais – pelo contrário, o *eval_loss* de ambos os modelos continua caindo de forma quase monotônica até a época 9–10, e a *execution accuracy* sobe junto (com ruído normal de treino, mas sem sinal de platô).

- **t5-small + RGAT:** `eval_loss` caiu 20% ($1,15 \rightarrow 0,92$), `exec` subiu $6\times$ ($0,68\% \rightarrow 4,10\%$) e `exact match` subiu $5\times$ ($1,04\% \rightarrow 5,19\%$).
- **t5-base + RGAT completo:** `eval_loss` caiu 13% ($1,68 \rightarrow 1,46$), `exec` subiu $8\times$ ($0,34\% \rightarrow 2,73\%$). `Exact match` permanece em 0% nas duas durações de treino.

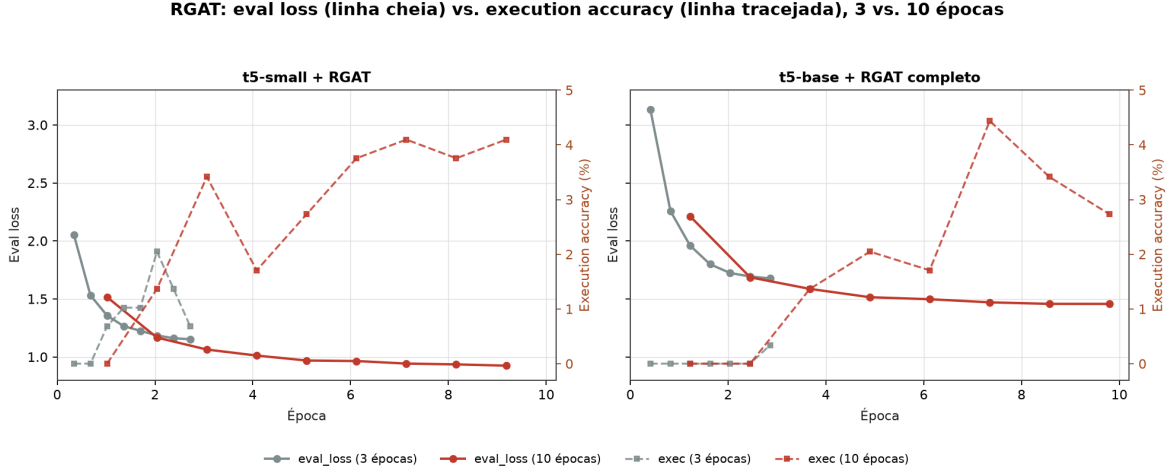


Figure 1: Eval loss (linha cheia, eixo esquerdo) e execution accuracy (linha tracejada, eixo direito) por época, 3 vs. 10 épocas, para os dois modelos com RGAT.

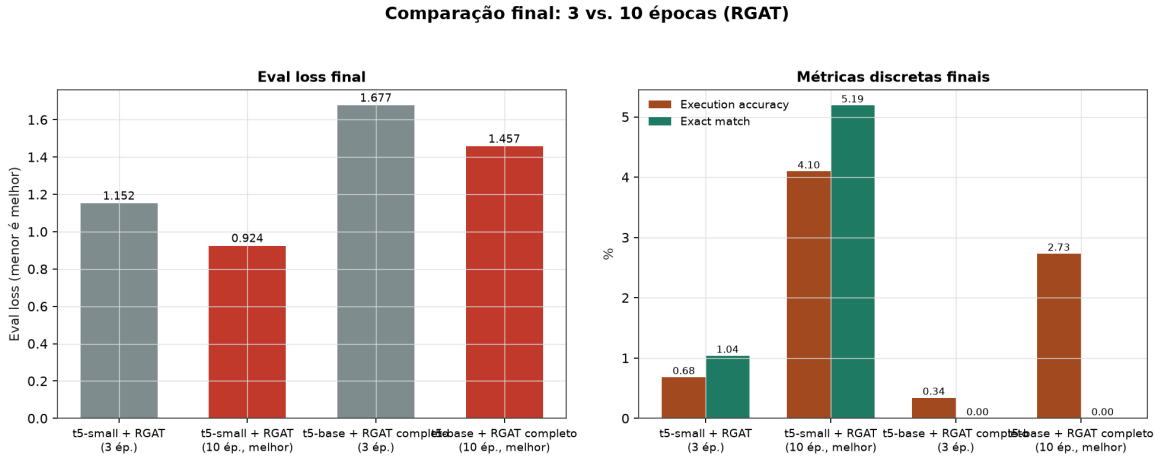


Figure 2: Comparação final (melhor checkpoint) entre as quatro rodadas.

3 Under- ou overfitting?

A Figura 1 mostra apenas *eval_loss* e *exec*. Para diagnosticar se o treino de 10 épocas cruzou de sub-treino para sobreajuste, é preciso comparar *eval_loss* com o *train_loss* (medido nos próprios lotes de treino) – a assinatura clássica de *overfitting* é *eval_loss* subindo enquanto *train_loss* continua caindo.

Nenhum dos dois modelos mostra *eval_loss* subindo – portanto, overfitting completo não ocorreu em nenhum caso. Mas o padrão difere entre os dois:

- **t5-small + RGAT:** *train_loss* achata por volta da época 2-3 e fica praticamente estável dali em diante, enquanto *eval_loss* continua caindo até o fim (1,51 → 0,92). O modelo de validação ainda melhora mesmo com o treino já estabilizado – sinal inequívoco de **underfitting**, sem ressalvas.
- **t5-base + RGAT completo:** *train_loss* cai de forma suave e contínua o treino inteiro (sem platô visível), mas *eval_loss* desacelera fortemente nas últimas épocas (1,497 → 1,470 → 1,457 → 1,4567 entre as épocas 6 e 10 – praticamente reto). Esse é o **início do padrão de overfitting** (*gap* entre as duas curvas se abrindo), embora ainda não configure overfitting completo, já que *eval_loss* nunca inverte e volta a subir.

Train loss vs. eval loss -- sem sinal de overfitting (nenhuma divergência)

(eixo Y cortado em 4,0 -- os primeiros passos do t5-base chegam a ~15,5 -- para focar na região relevante da comparação)

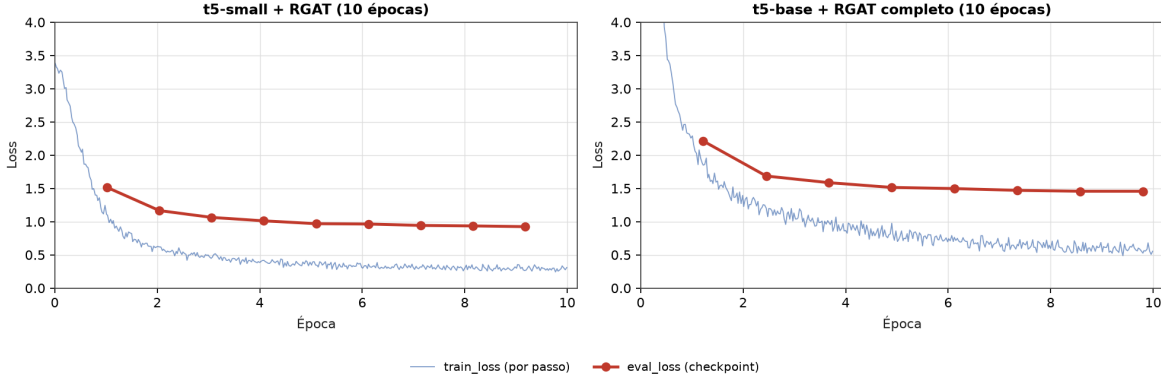


Figure 3: Train loss (linha fina, por passo) vs. eval loss (linha grossa, por checkpoint), 10 épocas.

Implicação prática: o t5-small provavelmente ainda se beneficiaria de mais épocas de treino; o t5-base+RGAT completo está no limiar – treiná-lo por mais tempo com os mesmos hiperparâmetros provavelmente teria retorno decrescente e arriscaria cruzar para overfitting, em vez de continuar fechando a lacuna para o T5 puro.

4 O que isso muda na conclusão anterior

A conclusão de `estudo/scibench_ablation.tex` – “o RGAT nunca vence sua contraparte do mesmo tamanho” – **precisa ser matizada, não descartada**. Dois pontos ficam mais claros agora:

1. Sub-treino era parte real da explicação. Com apenas 3 épocas, o RGAT estava genuinamente longe de convergir: suas camadas de atenção sobre grafo, inicializadas do zero, exigem mais passos de gradiente que o backbone T5 pré-treinado para chegar perto de um bom ponto de operação. Isso não era visível na comparação de 3 épocas porque não havia uma segunda duração de treino para contrastar – descartar essa hipótese sem testá-la teria sido prematuro.

2. Mesmo com 10 épocas, o RGAT continua atrás do T5 puro. Comparando com o t5-base sem RGAT (Tabela 1 de `estudo/scibench_ablation.tex`: exec 8,19%, treinado em apenas 3 épocas), o t5-base + RGAT completo com 10 épocas (exec 2,73%) ainda fica bem atrás – e treinou mais que o *dobro* do tempo (16h13 vs. 4h13) para chegar lá. Ou seja: mais treino claramente ajuda o RGAT, mas não fecha a lacuna para o T5 puro dentro do orçamento testado. Não podemos afirmar que o RGAT alcançaria ou superaria o T5 puro com treino ainda mais longo – apenas que a comparação de 3 épocas subestimava o potencial do RGAT, e que a arquitetura continua exigindo mais recursos computacionais por unidade de ganho do que o T5 puro nesta tarefa e neste regime de dados.

Conclusão revisada: “RGAT precisa de mais treino” é **confirmado** como fator real – não é apenas uma hipótese descartável. Mas “RGAT supera T5 puro” continua **não confirmado** mesmo com o triplo de épocas; o gargalo observado originalmente é uma combinação de sub-treino (agora corrigido em parte) e um custo-benefício computacional que ainda favorece o T5 puro neste conjunto de experimentos.

5 Log de validação, por configuração

As quatro tabelas abaixo listam, em ordem cronológica, todos os *checkpoints* de avaliação (*eval_loss*, *exact match*, *exec*) registrados durante o treino de cada configuração, extraídos diretamente de `trainer_state.json`.

Table 2: t5-small + RGAT, 3 épocas (graphix-scibench-t5small-rgat).

Época	eval_loss	exact match	exec
0,34	2,0544	0,00%	0,00%
0,68	1,5306	0,00%	0,00%
1,02	1,3561	0,35%	0,68%
1,36	1,2649	1,04%	1,02%
1,70	1,2252	1,04%	1,02%
2,04	1,1843	1,38%	2,05%
2,38	1,1609	1,38%	1,37%
2,72	1,1517	1,04%	0,68%

Table 3: t5-small + RGAT, 10 épocas (graphix-scibench-t5small-rgat-10ep).

Época	eval_loss	exact match	exec
1,02	1,5148	0,00%	0,00%
2,04	1,1662	0,69%	1,37%
3,06	1,0633	1,73%	3,41%
4,08	1,0114	2,08%	1,71%
5,10	0,9687	2,77%	2,73%
6,12	0,9640	3,46%	3,75%
7,14	0,9423	5,54%	4,10%
8,16	0,9350	4,50%	3,75%
9,18	0,9243	5,19%	4,10%

Table 4: t5-base + RGAT completo, 3 épocas (graphix-scibench-finetune-gc).

Época	eval_loss	exact match	exec
0,41	3,1330	0,00%	0,00%
0,82	2,2575	0,00%	0,00%
1,22	1,9604	0,00%	0,00%
1,63	1,7988	0,00%	0,00%
2,04	1,7232	0,00%	0,00%
2,45	1,6932	0,00%	0,00%
2,86	1,6765	0,00%	0,34%

Table 5: t5-base + RGAT completo, 10 épocas (graphix-scibench-finetune-gc-10ep).

Época	eval_loss	exact match	exec
1,22	2,2130	0,00%	0,00%
2,45	1,6860	0,00%	0,00%
3,67	1,5863	0,00%	1,37%
4,90	1,5142	0,00%	2,05%
6,12	1,4971	0,00%	1,71%
7,35	1,4702	0,35%	4,44%
8,57	1,4570	0,00%	3,41%
9,80	1,4567	0,00%	2,73%